МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ 1

з дисципліни “Функціональне програмування”

на тему « Основи функціонального програмування у Python »

Виконав:

Студент 2 курсу

групи ФеП-23

МАГОРА Максим

Перевірив:

СВАЛЕБА Сергій Андрійович

з дисципліни «Функціональне програмування»

Тема: Основи функціонального програмування у Python

PDFi ще 1

Варіант № 11: Мобільний тариф

PDF

Виконав: Студент 2 курсу групи ФеП-23 МАГОРА Максим

PDF

Перевірив: СВАЛЕБА Сергій Андрійович

PDF

1. Мета та завдання роботи

- Зрозуміти, що таке чиста функція і референтна прозорість.

  DOCX

- Навчитися виявляти та ізолювати побічні ефекти (I/O, глобальний стан, мутації).

  DOCX

- Переписати імперативний фрагмент коду у функціональному стилі.

  DOCX

- Застосувати `typing.Callable` для параметризації обчислень, фільтрації та політик бонусів.

  DOCX

- Інтегрувати специфічні ключові слова варіанта: `mobile`, `tariff`, `minutes`, `sms`, `traffic`, `data`, `bonus`, `monthly`, `cost`, `limit`, `filter`.

2. Структура проєкту та модулів (Варіант № 11)

Відповідно до архітектурного підходу методичних вказівок, проєкт має таку структуру папок і модулів:

PDFі ще 2

Plaintext

```
lab01/
├── core.py             # Чисті функції предметної області
├── app.py              # Оболонка побічних ефектів та I/O[cite: 7]
├── tests/
│   └── test_core.py    # Pytest-тести[cite: 7]
└── README.md           # Опис виконаної роботи та інструкція із запуску[cite:
7]
```

PDF

3. Опис і назви методів (функцій), реалізованих у проєкті

Відповідно до методичних вказівок, для Варіанта 11 («Мобільний тариф») реалізовано такі модулі та функції:

PDF

A. Чисте ядро (`core.py`)

- **`calculate_extra_minutes_cost(record: UsageRecord) -> float`**

  Опис: Чиста детермінована функція розрахунку понаднормових хвилин (`minutes`) понад ліміт (`limit`).

  PDF

- **`calculate_extra_sms_cost(record: UsageRecord) -> float`**

  Опис: Чиста функція розрахунку понаднормових SMS-повідомлень (`sms`).

  PDF

- **`calculate_extra_gb_cost(record: UsageRecord) -> float`**

  Опис: Чиста функція розрахунку понаднормового інтернет-трафіку та даних (`traffic`, `data`, `gb`).

  PDF

- **`compose(f: Callable[[B], C], g: Callable[[A], B]) -> Callable[[A], C]`**

  Опис: Функція композиції для побудови складніших правил із простіших блоків.

  DOCX

- **`make_loyalty_bonus_policy(discount_percent: float) -> BonusFn`**

Опис: Фабрика функцій для генерації політики бонусів/знижок (`bonus`, `tariff`) через `Callable`.

PDFi ще 1

- **`with_monthly_cost(record: UsageRecord, ...) -> ProcessedUsageRecord`**

  Опис: Аналог функції `with_total` з методички — повертає **новий словник** (створює новий об'єкт), не мутуючи вхідні дані.

  PDFi ще 3

- **`make_tariff_processor(*, accept: FilterFn, bonus_policy: BonusFn) -> Callable[...]`**

  Опис: Функція вищого порядку (фабрика обробника), що приймає правила фільтрації (`filter`, `limit`) та політики обчислень через `typing.Callable`.

  PDFi ще 3

Б. Оболонка побічних ефектів (`app.py`)

- **`render_report(result: dict[str, object]) -> None`**

  Опис: Функція виведення (`I/O`, `print`), яка приймає чистий результат роботи ядра та відображає звіт у консолі[cite: 7].

- **`run() -> None`**

  Опис: Точка входу в програму, що ініціалізує початкові дані (`mobile`, `tariff`) та запускає обробку.

  PDFi ще 1

4. Висновки

Під час виконання лабораторної роботи №1 (варіант 11 — «Мобільний тариф») було практично закріплено принципи функціонального програмування на мові Python:

PDF

1. **Забезпечення чистоти функцій та референтної прозорості:** Усі розрахункові функції (`calculate_extra_minutes_cost`, `calculate_extra_sms_cost`, `calculate_extra_gb_cost`) є повністю детермінованими, завжди повертають однаковий результат за однакових вхідних аргументів і не залежать від зовнішнього або глобального стану.

PDF

2. **Ізоляція побічних ефектів:** Усі операції з введення-виведення даних (`print`) винесено з обчислювального ядра в окремий модуль оболонки (`app.py` у функцію `render_report`). Завдяки функції `with_monthly_cost`, яка замість зміни існуючих словників формує та повертає нові об'єкти, вдалося повністю уникнути мутацій вхідних даних.

   PDFі ще 1

3. **Параметризація обчислень через `typing.Callable`:** Використання функцій вищого порядку (таких як `make_tariff_processor` та фабрики `make_loyalty_bonus_policy`) дозволило відокремити загальну схему обробки колекції даних від конкретних бізнес-правил (фільтрації лімітів та нарахування бонусів/знижок). Це забезпечило високий рівень модульності та гнучкості.

   PDFі ще 1

4. **Тестування:** За допомогою модульних `pytest`-тестів (`test_referential_transparency`, `test_no_mutation`, `test_callable_policies`) було автоматично перевірено стабільність роботи чистого ядра, відсутність побічних ефектів у вигляді мутацій та коректність застосування переданих через `Callable` політик.

   PDF